NATIONAL SPACE HACKATHON 2026

Orbital Debris Avoidance & Constellation Management System

Problem Statement

Hosted by Indian Institute of Technology, Delhi

Contents

1 Background **2**

2 Core Objectives **3**

3 Physics, Coordinate Systems, and Orbital Mechanics **4**
3.1 Reference Frames and State Vectors 4
3.2 Orbital Propagation Models 4
3.3 Conjunction Thresholds 4

4 API Specifications and Constraints **5**
4.1 Telemetry Ingestion API 5
4.2 Maneuver Scheduling API 5
4.3 Simulation Fast-Forward (Tick) API 6

5 Detailed Maneuver & Navigation Logic **7**
5.1 Propulsion Constraints and Fuel Mass Depletion 7
5.2 The Station-Keeping Box 7
5.3 Maneuver Vectors: The RTN Frame 8
5.4 Communication Latency and Blackout Zones 8
5.5 Provided Datasets 8
5.5.1 Ground Station Network (`ground_stations.csv`) 8

6 Frontend: The "Orbital Insight" Visualizer **9**
6.1 Performance Constraints 9
6.2 Required Visualization Modules 9
6.3 Visualization API Integration 10

7 Evaluation Criteria **11**

8 Deployment Requirements **11**

9 Expected Deliverables **11**

1 Background

Over the past decade, Low Earth Orbit (LEO) has transformed from a vast frontier into a highly congested orbital highway. The rapid deployment of commercial mega-constellations has exponentially increased the number of active payloads. Alongside these operational satellites, millions of pieces of space debris—ranging from defunct rocket bodies and shattered solar panels to stray bolts—orbit the Earth at hypervelocity speeds exceeding 27,000 km/h.

This severe congestion brings us perilously close to the **Kessler Syndrome**, a theoretical scenario proposed by NASA scientist Donald Kessler. In this scenario, the density of objects in LEO becomes high enough that a single collision generates a cloud of shrapnel, triggering a cascading chain reaction of further collisions. Because kinetic energy scales with the square of velocity, even a collision with a centimeter-sized fragment can completely destroy a satellite and instantly generate thousands of new trackable debris pieces.

Currently, satellite collision avoidance is a heavily manual, human-in-the-loop process. Ground-based radar networks, such as the US Space Surveillance Network (SSN), track large debris and issue **Conjunction Data Messages (CDMs)** when a close approach is predicted. Flight Dynamics Officers (FDOs) on Earth must manually evaluate these warnings, calculate the necessary orbital perturbations, and uplink thruster maneuver commands.

However, this legacy approach suffers from critical bottlenecks that make it unsustainable for the future of spaceflight:

- **Scalability Limits:** Manual evaluation cannot scale to handle constellations comprising thousands of satellites, which may collectively face hundreds of conjunction warnings daily.
- **Communication Latency & Blackouts:** Satellites frequently pass through "blackout zones" (such as over deep oceans) where no ground station has line-of-sight. If a conjunction is predicted while a satellite is out of contact, ground control is entirely helpless.
- **Suboptimal Resource Management:** Fuel in space is a finite, non-replenishable resource. Human operators struggle to globally optimize fuel consumption (Δv) across an entire fleet while simultaneously ensuring satellites return to their assigned orbital slots to maintain mission uptime.

The Challenge: The space industry requires a paradigm shift from ground-reliant piloting to onboard autonomy. Your task is to design an **Autonomous Constellation Manager (ACM)**. You must develop a robust, high-performance software suite capable of ingesting high-volume orbital telemetry, predicting conjunctions efficiently without $O(N^2)$ bottlenecks, and autonomously executing optimal evasion and return maneuvers.

2 Core Objectives

The primary objective of this hackathon is to architect, develop, and deploy an **Autonomous Constellation Manager (ACM)**. This backend system will act as a centralized, high-performance "brain" for a fleet of over 50 active satellites, navigating a hazardous environment populated by tens of thousands of tracked space debris fragments.

Participants must move beyond simple reactive scripting to build a system capable of predictive modeling, spatial optimization, and automated decision-making. Your ACM must successfully handle the following core responsibilities:

- **High-Frequency Telemetry Ingestion:** Your system must establish a robust pipeline to continuously process incoming orbital state vectors—specifically, position ($\vec{r}$) and velocity ($\vec{v}$) in the Earth-Centered Inertial (ECI) coordinate frame. This data stream will represent the real-time kinematic states of both your controlled constellation and the uncontrolled debris field.

- **Predictive Conjunction Assessment (CA):** The software must forecast potential collisions (Conjunction Data Messages) up to 24 hours in the future. Because checking every satellite against every piece of debris is an $O(N^2)$ operation, participants must implement highly efficient spatial indexing algorithms to calculate the Time of Closest Approach (TCA) without exceeding computational or time constraints.

- **Autonomous Collision Avoidance (COLA):** When a critical conjunction (a miss distance of < 100 meters) is predicted, the system must autonomously calculate and schedule an evasion maneuver. This involves determining the optimal burn window and the exact Δv (change in velocity) vector required to push the satellite to a safe standoff distance, factoring in thruster cooldowns and orbital mechanics.

- **Station-Keeping and Orbital Recovery:** A satellite is only useful when it is in its assigned mission slot. Evasion maneuvers will inherently perturb the satellite's orbit. The ACM must calculate and execute a subsequent "recovery burn" to correct the orbital drift and return the payload to its designated spatial bounding box (station-keeping) as quickly as possible.

- **Propellant Budgeting & End-of-Life (EOL) Management:** Spacecraft cannot refuel. Every burn depletes the finite propellant mass (m_{fuel}), governed by the Tsiolkovsky rocket equation. Your software must track these fuel budgets strictly. If a satellite's fuel reserves drop to a critical threshold (e.g., 5%), the system must preemptively schedule a final maneuver to move it into a safe "graveyard orbit," preventing it from becoming dead, uncontrollable debris itself.

- **Global Multi-Objective Optimization:** The ultimate algorithmic challenge is balancing two directly opposing metrics: maximizing **Constellation Uptime** (the total time satellites spend actively performing their mission in their assigned slots) while minimizing the total **Fuel Expenditure** across the fleet.

3 Physics, Coordinate Systems, and Orbital Mechanics

To accurately simulate the orbital environment and evaluate the validity of your collision avoidance maneuvers, your physics engine must adhere to strict mathematical and physical frameworks.

3.1 Reference Frames and State Vectors

All kinematic data in this simulation is grounded in the **Earth-Centered Inertial (ECI)** coordinate system (J2000 epoch). The ECI frame is non-rotating relative to the stars, making it the standard for calculating orbital trajectories without the fictitious forces (Coriolis and centrifugal) present in Earth-Centered, Earth-Fixed (ECEF) frames.

Every object in the simulation (satellites and debris) is defined by a 6-dimensional **State Vector** at a given time t:

$$S(t) = \begin{bmatrix} \vec{r}(t) \\ \vec{v}(t) \end{bmatrix} = \begin{bmatrix} x, y, z, v_x, v_y, v_z \end{bmatrix}^T$$

where position $\vec{r}$ is in kilometers (km) and velocity $\vec{v}$ is in kilometers per second (km/s).

3.2 Orbital Propagation Models

Participants cannot assume simple, unperturbed two-body Keplerian orbits. Due to the equatorial bulge of the Earth, orbits experience nodal regression and apsidal precession. Your propagation engine must, at a minimum, account for the **J2 perturbation**. The equations of motion governing a satellite are given by the second-order ordinary differential equation:

$$\frac{d^2\vec{r}}{dt^2} = -\frac{\mu}{|\vec{r}|^3}\vec{r} + \vec{a}_{J2}$$

Where $\mu = 398600.4418$ km^3/s^2 is the Earth's standard gravitational parameter, and the J2 acceleration vector $\vec{a}_{J2}$ is defined as:

$$\vec{a}_{J2} = \frac{3}{2}\frac{J_2 \mu R_E^2}{|\vec{r}|^5} \begin{bmatrix} x\left(5\frac{z^2}{|\vec{r}|^2} - 1\right) \\ y\left(5\frac{z^2}{|\vec{r}|^2} - 1\right) \\ z\left(5\frac{z^2}{|\vec{r}|^2} - 3\right) \end{bmatrix}$$

(Assume $R_E = 6378.137$ km and $J_2 = 1.08263 \times 10^{-3}$). You are expected to use robust numerical integration methods (e.g., Runge-Kutta 4th Order) to propagate these states forward in time.

3.3 Conjunction Thresholds

A collision is defined mathematically when the Euclidean distance between a satellite ($\vec{r}_{sat}$) and any debris object ($\vec{r}_{deb}$) falls below the critical threshold D_{crit}:

$$|\vec{r}_{sat}(t) - \vec{r}_{deb}(t)| < 0.100 \text{ km} \quad (100 \text{ meters})$$

4 API Specifications and Constraints

Your Autonomous Constellation Manager must expose a robust RESTful API on port `8000`. The simulation engine will communicate with your software exclusively through these endpoints.

4.1 Telemetry Ingestion API

This endpoint will be flooded with high-frequency state vector updates. Your system must parse this data and asynchronously update its internal physics state.
Endpoint: `POST /api/telemetry`
Request Body:

```
{
  "timestamp": "2026-03-12T08:00:00.000Z",
  "objects": [
    {
      "id": "DEB-99421",
      "type": "DEBRIS",
      "r": {"x": 4500.2, "y": -2100.5, "z": 4800.1},
      "v": {"x": -1.25, "y": 6.84, "z": 3.12}
    }
  ]
}
```

Response (200 OK):

```
{
  "status": "ACK",
  "processed_count": 1,
  "active_cdm_warnings": 3
}
```

4.2 Maneuver Scheduling API

When your system calculates an evasion or recovery burn, it must submit the maneuver sequence here. The simulation will validate the line-of-sight constraints, apply the Δv instantaneously at the specified `burnTime`, and deduct the corresponding fuel mass.
Endpoint: `POST /api/maneuver/schedule`
Request Body:

```
{
  "satelliteId": "SAT-Alpha-04",
  "maneuver_sequence": [
    {
      "burn_id": "EVASION_BURN_1",
      "burnTime": "2026-03-12T14:15:30.000Z",
      "deltaV_vector": {"x": 0.002, "y": 0.015, "z": -0.001}
    },
    {
      "burn_id": "RECOVERY_BURN_1",
      "burnTime": "2026-03-12T15:45:30.000Z",
      "deltaV_vector": {"x": -0.0019, "y": -0.014, "z": 0.001}
    }
  ]
}
```

Response (202 Accepted):

```
{
  "status": "SCHEDULED",
  "validation": {
    "ground_station_los": true,
    "sufficient_fuel": true,
    "projected_mass_remaining_kg": 548.12
  }
}
```

4.3 Simulation Fast-Forward (Tick) API

To test your system's efficiency, the grader will advance the simulation time by arbitrary steps. During this "tick", your engine must integrate the physics for all objects and execute any maneuvers scheduled within that time window.

Endpoint: `POST /api/simulate/step`

Request Body:

```
{
  "step_seconds": 3600
}
```

Response (200 OK):

```
{
  "status": "STEP_COMPLETE",
  "new_timestamp": "2026-03-12T09:00:00.000Z",
  "collisions_detected": 0,
  "maneuvers_executed": 2
}
```

5 Detailed Maneuver & Navigation Logic

The core algorithmic challenge of Project AETHER lies in calculating and executing evasion and recovery maneuvers. Your system cannot simply "teleport" satellites out of harm's way; it must obey the strict physical constraints of spacecraft propulsion and orbital mechanics.

5.1 Propulsion Constraints and Fuel Mass Depletion

Every satellite in the constellation is identical, utilizing a monopropellant chemical thruster system. You must assume **impulsive burns**, meaning the change in velocity ($\Delta\vec{v}$) is applied instantaneously, altering the velocity vector without changing the position vector at the exact moment of the burn.

The spacecraft physical constants are defined as follows:

- **Dry Mass (m_{dry}):** 500.0 kg
- **Initial Propellant Mass (m_{fuel}):** 50.0 kg (Total initial wet mass = 550.0 kg)
- **Specific Impulse (I_{sp}):** 300.0 s
- **Maximum Thrust Limit:** $|\Delta\vec{v}| \leq 15.0$ m/s per individual burn command.
- **Thermal Cooldown:** A mandatory 600-second rest period is required between any two burns on the same satellite to prevent thruster degradation.

Your simulation must rigidly track the mass of each satellite. Following the Tsiolkovsky rocket equation, the mass of propellant consumed (Δm) for a given maneuver is calculated as:

$$\Delta m = m_{\text{current}} \left(1 - e^{-\frac{|\Delta\vec{v}|}{I_{sp} \cdot g_0}}\right)$$

Where $g_0 = 9.80665$ m/s^2 (standard gravity). *Note: As fuel is depleted, the satellite becomes lighter, making subsequent maneuvers slightly more fuel-efficient. Your API must dynamically account for this mass change.*

5.2 The Station-Keeping Box

Satellites are deployed to provide continuous coverage over specific geographic regions. Therefore, each satellite is assigned a **Nominal Orbital Slot**—a dynamic reference point propagating along the ideal, unperturbed orbit.

- **Drift Tolerance:** A satellite is considered "Nominal" as long as its true position remains within a **10 km spherical radius** of its designated slot.
- **Uptime Penalty:** If a collision avoidance maneuver pushes the satellite outside this bounding box, the system logs a *Service Outage*. Your **Uptime Score** degrades exponentially for every second spent outside the box.
- **Recovery Burn Requirement:** Every evasion maneuver must be paired with a calculated recovery trajectory (e.g., a phasing orbit or Hohmann transfer) to return the satellite to its slot once the debris threat has safely passed.

5.3 Maneuver Vectors: The RTN Frame

While state vectors are provided in the global ECI frame, maneuver planning is typically calculated in the satellite's local **Radial-Transverse-Normal (RTN)** coordinate frame:

- **Radial (R):** Points from the Earth's center through the satellite. A *Radial Shunt* alters eccentricity and the argument of perigee.
- **Transverse (T):** Points in the direction of velocity, perpendicular to R. A *Prograde/Retrograde* burn (Phasing Maneuver) is the most fuel-efficient way to change the semi-major axis and orbital period, allowing the satellite to "speed up" or "slow down" relative to the debris.
- **Normal (N):** Orthogonal to the orbital plane ($\vec{R} \times \vec{T}$). A *Plane Change* burn alters inclination and the Right Ascension of the Ascending Node (RAAN). **Warning:** Out-of-plane maneuvers are notoriously fuel-expensive and should be avoided unless absolutely necessary.

Participants must calculate the required $\Delta\vec{v}$ in the RTN frame and apply the appropriate rotation matrix to convert the thrust vector back into the ECI frame before submitting the command to the API.

5.4 Communication Latency and Blackout Zones

Satellites are not constantly connected to Mission Control. Your Autonomous Constellation Manager is assumed to be running on ground servers.

- **Line-of-Sight (LOS) Requirement:** A maneuver command can only be successfully transmitted if the target satellite has an unobstructed geometric line-of-sight to at least one active Ground Station, taking into account the Earth's curvature and the station's minimum elevation mask angle.
- **Signal Delay:** There is a hardcoded 10-second latency for any API command. You cannot schedule a burn to occur earlier than Current Simulation Time + 10 seconds.
- **Blind Conjunctions:** If a collision is predicted to occur over an ocean or pole (a blackout zone), your system must possess the predictive capability to schedule and upload the evasion sequence *before* the satellite leaves the coverage area of the last available ground station.

5.5 Provided Datasets

Participants will be provided with several starting datasets to initialize their physics engines.

5.5.1 Ground Station Network (`ground_stations.csv`)

To calculate communication blackouts and maneuver upload windows, your system must check line-of-sight against the provided ground station network. A satellite can only receive commands if it is above the `Min_Elevation_Angle_deg` of at least one station.

```
Station_ID,Station_Name,Latitude,Longitude,Elevation_m,
   Min_Elevation_Angle_deg
GS-001,ISTRAC_Bengaluru,13.0333,77.5167,820,5.0
GS-002,Svalbard_Sat_Station,78.2297,15.4077,400,5.0
GS-003,Goldstone_Tracking,35.4266,-116.8900,1000,10.0
GS-004,Punta_Arenas,-53.1500,-70.9167,30,5.0
GS-005,IIT_Delhi_Ground_Node,28.5450,77.1926,225,15.0
GS-006,McMurdo_Station,-77.8463,166.6682,10,5.0
```

6 Frontend: The "Orbital Insight" Visualizer

While the backend physics engine handles the heavy numerical computations, situational awareness is paramount for human-in-the-loop oversight. Teams must build a **2D Operational Dashboard** analogous to the software utilized by Flight Dynamics Officers (FDOs) at mission control.

6.1 Performance Constraints

The visualizer must be capable of rendering **50+ active satellites** and **10,000+ debris objects** in real-time. Standard DOM manipulation will severely bottleneck the browser; therefore, the use of the **Canvas API** or **WebGL** (via libraries such as PixiJS, Deck.gl, or Three.js) is highly recommended to maintain a stable 60 Frames Per Second (FPS).

6.2 Required Visualization Modules

Your frontend dashboard must incorporate the following distinct modules:

- **The "Ground Track" Map (Mercator Projection):** A dynamic 2D world map displaying the sub-satellite points over the Earth's surface. It must feature:
  - Real-time location markers for the entire active constellation.
  - A historical trailing path representing the last 90 minutes of orbit.
  - A dashed predicted trajectory line for the next 90 minutes.
  - A dynamic shadow overlay representing the "Terminator Line" (the boundary between day and night) to indicate solar eclipse zones where satellites must rely on battery power.
- **The Conjunction "Bullseye" Plot (Polar Chart):** A relative proximity view of debris approaching a selected satellite.
  - **Center Point:** The selected satellite is fixed at the origin.
  - **Radial Distance:** Represents the *Time to Closest Approach* (TCA).
  - **Angle:** Represents the relative approach vector.
  - **Risk Indexing:** Debris markers must be color-coded based on the probability of collision and miss distance (e.g., Green = Safe, Yellow = Warning < 5 km, Red = Critical < 1 km).
- **Telemetry & Resource Heatmaps:** Fleet-wide health monitoring. This includes a visual fuel gauge representing m_{fuel} for every satellite and a Δv cost analysis graph plotting "Fuel Consumed" versus "Collisions Avoided" to visually demonstrate the efficiency of your evasion algorithms.
- **The Maneuver Timeline (Gantt Scheduler):** A chronological schedule of past and future automated actions. It must display distinct blocks for "Burn Start," "Burn End," and the mandatory 600-second thruster cooldowns, clearly flagging any conflicting commands or blackout zone overlaps.

6.3 Visualization API Integration

To support this high-density frontend without overwhelming the network, your API must include a highly optimized `Snapshot` endpoint.
Endpoint: `GET /api/visualization/snapshot`
Response (200 OK):

```
{
  "timestamp": "2026-03-12T08:00:00.000Z",
  "satellites": [
    {
      "id": "SAT-Alpha-04",
      "lat": 28.545,
      "lon": 77.192,
      "fuel_kg": 48.5,
      "status": "NOMINAL"
    }
  ],
  "debris_cloud": [
    ["DEB-99421", 12.42, -45.21, 400.5],
    ["DEB-99422", 12.55, -45.10, 401.2]
  ]
}
```

Note: The `debris_cloud` *array should utilize a flattened or tuple-based structure (e.g.,* `[ID, Latitude, Longitude, Altitude]`*) to drastically compress the JSON payload size for rapid network transfer.*

7 Evaluation Criteria

The hackathon will employ a rigorous two-phase evaluation process. Phase 1 consists of an automated objective assessment where your physics engine and API will be stress-tested against thousands of simulated debris objects. Phase 2 involves a manual evaluation of the frontend and code architecture by our judging panel.

Criteria	Weightage	Description
Safety Score (Objective)	25%	Percentage of conjunctions successfully avoided. A single collision (miss distance < 100m) results in severe penalty points.
Fuel Efficiency (Objective)	20%	Total Δv consumed across the constellation. Evaluates the mathematical optimization of your evasion algorithms.
Constellation Uptime	15%	Measures the total time satellites spend within 10 km of their Nominal Orbital Slots.
Algorithmic Speed	15%	Time complexity of the backend API. Your code must maintain high performance while calculating spatial indices and numerical integrations.
UI/UX & Visualization	15%	Evaluates the clarity, frame rate, and situational awareness provided by the Orbital Insight dashboard.
Code Quality & Logging	10%	Assesses modularity, documentation, and the accuracy of the system's maneuver logging capabilities.

8 Deployment Requirements

Note: This is a hard requirement! If the repository provided does not build using Docker and does not use the specified base image, your submission cannot be auto-tested and will be disqualified.

- **Docker Initialization:** There must be a `Dockerfile` at the root of your GitHub repository that initializes the simulation and exposes the required APIs.
- **Base Image:** The Dockerfile must use the `ubuntu:22.04` image. This prevents dependency conflicts and ensures cross-environment consistency during automated grading.
- **Port Binding:** Port `8000` must be exported so that the grading scripts can hit your API endpoints. Ensure your application binds to `0.0.0.0` and not just `localhost`.

9 Expected Deliverables

By the final submission deadline, teams must provide the following:

1. **Github Repo Link:** Link to your github repo containing your complete application (Backend + Frontend + Database). Make sure this repo is public.
2. **Docker Environment:** A valid `Dockerfile` at the root of the repository as specified above.

3. **Technical Report:** A brief PDF document (preferably in LaTeX) detailing the numerical methods, spatial optimization algorithms, and overall architecture used in your solution.
4. **Video Demonstration:** A video demo (under 5 minutes) showcasing th0e Idea, Implementation, Orbital Insight frontend and its core functionalities.

The official Submission Form for these deliverables will be released closer to the deadline.